

JavaScript Learning Notes

Ab tak jo topics cover kiye hain — concepts + important examples

1. Variables

let aur const variables banane ke liye use hote hain. var purana syntax hai.

```
let name = "Nitin";
const age = 25;
let city = "Panchkula";
```

2. Data Types

String, Number, Boolean, Undefined, Null, Object aur Array.

```
let name = "Nitin";    // string
let age = 25;          // number
let active = true;     // boolean
let x;                 // undefined
let y = null;          // null
let user = {name:"Nitin"}; // object
let nums = [10, 20, 30]; // array
```

3. Operators & Conditions

Arithmetic operators: +, -, *, /, %. Comparison: ===, !==, >, <, >=, <=, if/else se conditions handle karte hain.

```
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

4. Functions

Function reusable code ka block hai. Parameters input lete hain aur return value wapas de sakta hai.

```
function add(a, b) {
  return a + b;
}

let result = add(10, 20);
console.log(result); // 30
```

5. Arrays & Loops

Array indexed collection hai. Index 0 se start hota hai. for aur for...of se iterate kar sakte hain.

```
let numbers = [10, 20, 30];

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}

for (let number of numbers) {
  console.log(number);
}
```

6. Array Methods

forEach = action/iteration; map = transform karke new array; filter = select items; reduce = multiple values ko one value mein combine.

```
let numbers = [10, 15, 20, 25, 30];

let doubled = numbers.map(function(number) {
  return number * 2;
});

let even = numbers.filter(function(number) {
  return number % 2 === 0;
});

let total = numbers.reduce(function(sum, number) {
  return sum + number;
}, 0);
```

7. Chaining

Methods ko chain kar sakte hain: filter → map → reduce.

```
let result = numbers
  .filter(function(number) {
    return number % 2 === 0;
  })
  .map(function(number) {
    return number * 2;
  })
  .reduce(function(sum, number) {
    return sum + number;
  }, 0);
```

8. Objects

Object key-value pairs store karta hai. Dot aur bracket notation se values access hoti hain. Properties update/add/delete bhi kar sakte hain.

```
let user = {
  name: "Nitin",
  age: 25,
  city: "Panchkula"
};

console.log(user.name);
user.age = 26;
user.email = "test@example.com";
delete user.city;
```

9. Nested Objects & Arrays of Objects

Object ke andar object aur array ke andar objects rakh sakte hain.

```
let user = {
  name: "Nitin",
  address: {
    city: "Panchkula",
    pin: 134109
  }
};

console.log(user.address.city);

let users = [
  {name: "A", age: 20},
  {name: "B", age: 25}
```

```
];  
  
console.log(users[0].name);
```

10. Object Methods

Object.keys(), Object.values(), Object.entries() useful built-in methods hain.

```
let product = {  
  name: "Phone",  
  price: 20000,  
  brand: "Samsung"  
};  
  
console.log(Object.keys(product));  
console.log(Object.values(product));  
console.log(Object.entries(product));  
  
Object.entries(product).forEach(function([key, value]) {  
  console.log(key, value);  
});
```

11. Destructuring

Array/object se values ko directly variables mein nikalna destructuring hai. Rename bhi kar sakte hain.

```
let colors = ["red", "green", "blue"];  
let [a, b, c] = colors;  
  
let user = {name: "Nitin", city: "Panchkula"};  
let {name, city} = user;  
  
let {name: userName} = user;  
  
function showProduct({name, price, brand}) {  
  console.log(name, price, brand);  
}
```

12. Spread Operator (...)

Spread existing array/object ki values ko expand/copy karta hai.

```
let fruits = ["Apple", "Mango"];  
let newFruits = [...fruits, "Banana"];  
  
let user = {name: "Nitin", age: 25};  
let updatedUser = {...user, age: 27};
```

13. Rest Operator (...)

Rest remaining values ko collect karta hai. Function arguments ko array mein collect karne ke liye bhi use hota hai.

```
function add(...numbers) {  
  return numbers.reduce(function(sum, n) {  
    return sum + n;  
  }, 0);  
}  
  
let [first, second, ...rest] = [10, 20, 30, 40];
```

14. DOM Basics

DOM se JavaScript HTML elements ko select aur change kar sakti hai.

```

let title = document.getElementById("title");
let btn = document.querySelector("#btn");

title.textContent = "Hello";
btn.addEventListener("click", function() {
    title.textContent = "Hello Nitin";
});

```

15. Input & Value

Input ki user-entered value input.value se milti hai. Number input bhi generally string value deta hai, isliye Number() useful hai.

```

let input = document.getElementById("ageInput");
let value = input.value;

if (value.trim() === "") {
    console.log("Please enter age");
} else {
    let age = Number(value);
    console.log(age);
}

```

16. Forms & Submit Event

Form submit par submit event use karte hain. event.preventDefault() page reload ko rokta hai. Validation output se pehle karna chahiye.

```

let form = document.getElementById("userForm");
let name = document.getElementById("name");
let email = document.getElementById("email");
let age = document.getElementById("age");
let result = document.getElementById("result");

form.addEventListener("submit", function(event) {
    event.preventDefault();

    if (
        name.value.trim() === "" ||
        email.value.trim() === "" ||
        age.value.trim() === ""
    ) {
        result.textContent = "Please fill all fields";
    } else {
        result.innerHTML =
            "Name: " + name.value + "<br>" +
            "Email: " + email.value + "<br>" +
            "Age: " + age.value;
    }
});

```

Quick Revision

map = change • filter = choose • reduce = combine • spread = expand • rest = collect • DOM = HTML ko JS se control karna

Next topic: DOM mein CSS/classes dynamically change karna — classList, style, show/hide — phir mini project.